

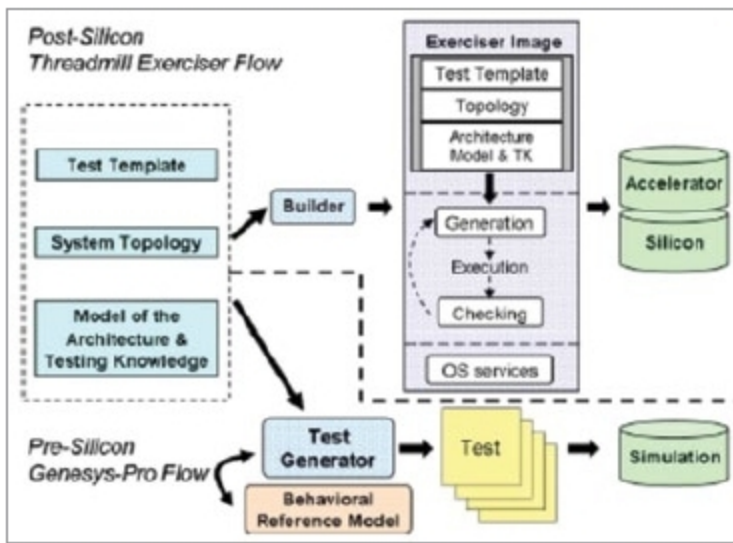


Fig. 16: Threadmill versus Genesys-Pro tool flows

disparate teams with complex flows and methodologies of their own, and include their unique design, implementation and validation phases. The challenge of security on observability is more direct. SoC designs include a large number of assets, such as cryptographic keys, DRM keys, firmware, debug mode and the like, which must be protected from unauthorised access.

Unfortunately, post-silicon observability and DfD infrastructure in silicon provide an obvious way to access such assets.

Further, much of the DfD infrastructure is available on-field to facilitate survivability. This permits their exploitation by malicious hackers to gain unauthorised access to system assets after deployment. Indeed, many celebrated system hacks have made use of post-silicon observability features, causing devastating impact to the product and company reputation once carried out. Consequently, a knee-jerk reaction is to restrict DfD features available in the design.

On the other hand, lack of DfD may make post-silicon validation difficult, long and even intractable. This may delay the product launch. With aggressive time-to-market requirement, a consequence of such delays can be a loss of billions of dollars in revenue or even missing the market for the product altogether.

Power management features also affect observability, but in a different manner. Power management features focus on turning off different hardware and software blocks at different points of execution, when not functionally necessary. The key problem is that observability requirements from debug

and validation are difficult to incorporate within the power management framework.

In particular, if a design block is in a low-power state, it is difficult to observe (or infer) the interaction of the block with other IPs in the SoC design. Lack of observability can affect debug of IPs different from the one subjected to power management.

For example, consider debugging IP A during a specific silicon execution. For this purpose, signals from A need to be routed to some observation point such as a memory or output pin.

Suppose, the routing includes IP B, which is in no way functionally dependent on A. It is possible then for B to be powered down during a part of the execution when A is active. However, this means that the route of observable signals from A is not active during that time, resulting in no observability of the internal behaviour of A.

One approach to address this challenge is to disable power management during silicon debug. However, this restricts the ability to debug and validate power management protocols themselves, for example, sequence of activities that must happen in order to transition an IP to different sleep (or wake-up) states.

Developing a post-silicon observability architecture that accounts for security and power management constraints is highly non-trivial. In particular, to comprehend the security/power-management/validation trade-offs, such an architecture must account for a number of other factors.

Planning. The primary goal of post-silicon validation is to identify design errors by exploiting the speed of post-silicon execution. It should be clarified that it is not necessary for post-silicon validation to completely diagnose or root cause a bug.

The goal is to narrow down from a post-silicon failure to an error scenario that can be effectively investigated in pre-silicon environment. Since a physical object (silicon) is involved in the validation process, the path is from an observed failure (for example,

system crash) to a resolution of the root cause for failure.

Test execution. This involves setting up the test environment and platform, running the test and, in case the test fails, performing some obvious sanity checks (like, checking if the SoC has been correctly set up on the platform, power sources are connected and switches are set up as expected for the test). If the problem is not resolved during sanity check, it is typically referred to as a pre-sighting.

Pre-sighting analysis. The goal of pre-sighting analysis is to make failure repeatable. This is highly non-trivial, since many failures occur under highly-subtle and coordinated execution of different IP blocks. For instance, IP A sends a message to IP C within a cycle of another IP B sending a different message to C. This may result in a buffer overflow (eventually resulting in a system crash), when occurring in a state in which input queue of C has only one slot left and before C has had the opportunity to remove some items from the queue.

Making failure repeatable requires running the test several times, under different software, hardware, systems and environmental conditions (possibly with some knowledge and experience on potential root causes) until a stable recipe for failure is discovered. At that point, failure is referred to as sighting.

Sighting disposition. Once a failure is confirmed as a sighting, a debug team is assigned for its disposition. This includes developing plans to track, address and create workarounds for the failure. The plan typically involves collaboration among representatives from architecture, design and implementation as well as personnel with expertise in specific design features exercised in the failing tests (for example, power management and secure boot).

Bug resolution. Once a plan of action has been developed for a sighting, it is referred to as a bug. A team is assigned for ensuring that it is resolved in a timely manner based on the plan. Resolution includes finding a workaround for the failure to enable exploration of other bugs and triaging, and identifying the root cause for the bug.

Triaging and root causing bugs are